DFSha: SISTEMA DE ARCHIVOS DISTRIBUIDO

Proyecto 1, fase I: diseño individual

ARQUITECTURA DE NUBE Y SISTEMAS DISTRIBUIDOS
FACULTAD DE INGENIERÍAS

ESTUDIANTE
MIGUEL MORENO

UNIVERSIDAD PONTIFICIA BOLIVARIANA
MEDELLÍN, ANTIOQUIA
2026

1. El servicio

El DFSha es un sistema de archivos distribuido donde cada usuario autenticado tiene su propio árbol de carpetas y puede guardar archivos grandes que no quedan en una sola máquina, sino partidos en bloques repartidos entre varios nodos, con copias para que el archivo siga disponible si un nodo se cae. La arquitectura asignada es la opción 1, cliente/servidor con un servicio distribuido entre servidores dentro de una red privada.

2. El diseño en pocas palabras

Un servidor de control sabe dónde está cada bloque y quién puede tocarlo, pero nunca mueve los datos, y unos nodos de datos (DataNodes) guardan los bloques y los copian entre ellos; el cliente le pregunta al control dónde están los bloques y luego va directo a los nodos, igual que alguien que mira el catálogo de una biblioteca y después va a cada estante.

3. Supuestos

- Se despliega en una VPC privada de AWS Academy, con instancias EC2 y Docker, y el cliente entra por VPN.
- Todos los bloques miden lo mismo, 4 MB, porque bloques de distinto tamaño complican ubicar los datos.
- Cada bloque tiene 3 copias (R = 3) y una escritura se acepta con 2 (W = 2), así que se necesitan mínimo 4 DataNodes.
- Cada usuario solo ve su espacio `/users/<usuario>` y no se comparten archivos.
- No se busca semántica POSIX completa; la de `read()` y `write()` se declara en la sección 8.

Estas son las simplificaciones que dejo declaradas para que el profesor las apruebe antes de implementar: los cambios se ven al cerrar el archivo (semántica de sesión) y no con cada `write()`; una modificación parcial reescribe el bloque completo de 4 MB; el último bloque se rellena hasta 4 MB; el lock solo es de escritura, porque los lectores no lo necesitan; no hay `truncate`, así que para acortar un archivo se reemplaza completo con `put`; no se admiten huecos, es decir, escribir más allá del final; solo se guarda la versión actual de cada archivo, sin historial; y no se comparten archivos entre usuarios.

4. Casos de uso

Hay tres actores: el usuario, que usa la consola `dfs`; el administrador, que agrega o retira nodos y crea usuarios; y los DataNodes, que reportan su estado al control.

Tabla 1

Operaciones del servicio

Req.	Operaciones	Para qué sirve	Quién participa
RF1	`ls, cd, mkdir, rmdir, rm, stat`	Navegar y organizar el árbol propio; `cd` solo cambia la carpeta actual del cliente después de validarla con `stat`.	Solo el control
RF2	`put` (send), `get` (recv)	Subir o bajar un archivo completo.	Control y varios DataNodes
RF3	`open, read, write, lock, unlock, close`	Leer o cambiar una parte del archivo sin transferirlo entero.	Control y los DataNodes del rango
Admin	`add-node, drain-node, add-user`	Crecer el almacenamiento o retirar un nodo de forma ordenada.	Control

Las secuencias de carga, lectura y escritura están en las figuras 3, 4 y 5. El borrado es más corto: el cliente envía `DELETE /fs/rm`, el control revisa que el archivo sea del usuario y que nadie lo tenga bloqueado (si no, responde `423`), registra el borrado en su journal, el archivo deja de verse de inmediato y responde `200`; después, en la respuesta a los heartbeats, les dice a los DataNodes que borren los bloques.

5. Arquitectura

Usé solo tres componentes, porque cada uno resuelve un requisito concreto, y por eso no hay un gateway aparte ni un broker de mensajes.

Tabla 2

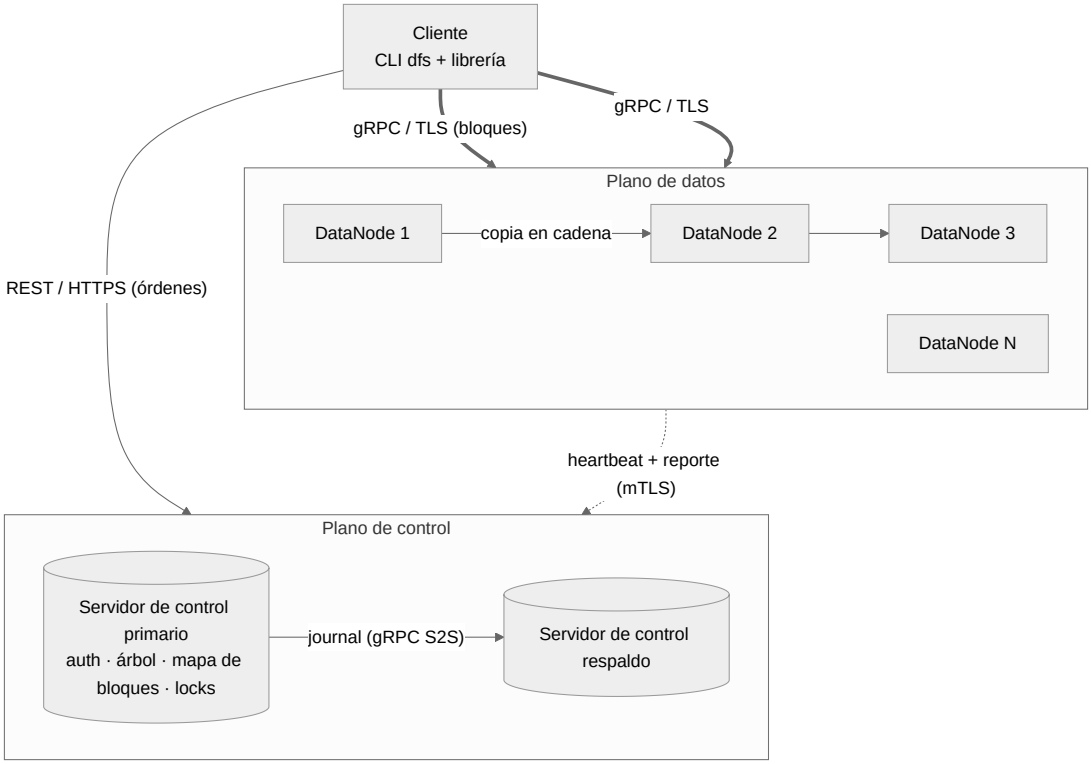
Componentes y responsabilidades

Componente	Plano	Qué hace
Cliente (consola y librería)	—	Se autentica, parte y une los archivos en bloques, calcula el SHA-256 de cada bloque, habla con varios DataNodes a la vez y, si uno falla, prueba con otra copia.
Servidor de control (primario y respaldo)	Control	Usuarios y permisos, árbol de carpetas, mapa archivo → bloques → copias, versiones, locks, elección de nodos, detección de fallos y reposición de copias.
DataNode (4 o más)	Datos	Guarda los bloques cifrados, revisa su hash, los pasa a la siguiente copia y envía heartbeats.

Separar el control de los datos evita que el servidor de control sea un cuello de botella, porque si todos los bytes pasaran por él, el sistema iría tan rápido como esa sola máquina; en cambio, así el cliente baja bloques de varios nodos al tiempo.

Figura 1

Diagrama de componentes

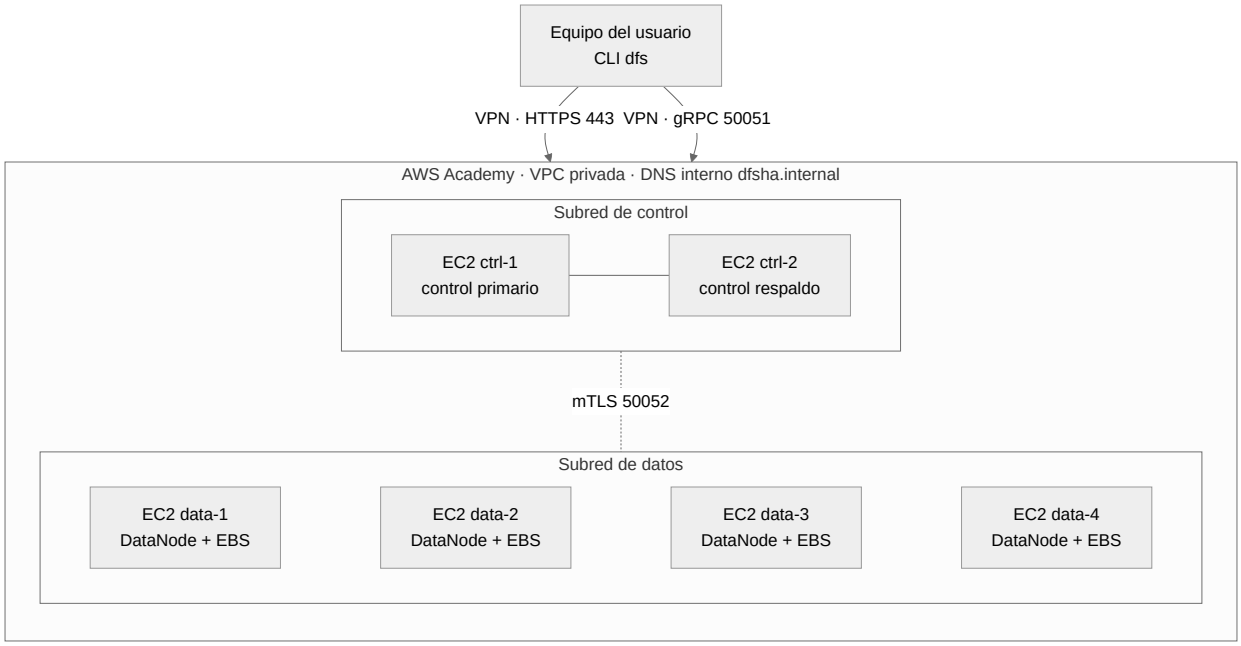


El control funciona como primario y respaldo: cada cambio se escribe primero en un journal (un registro de cambios) que se copia al respaldo antes de responder, y si el respaldo deja de recibir los pings del primario por 15 segundos, toma su lugar, sube un número de época y actualiza el nombre `control.dfsha.internal` en el DNS interno. Los DataNodes rechazan órdenes con una época vieja, así que un primario que vuelva no puede seguir mandando.

Para que los dos no terminen mandando a la vez cuando solo se corta la red entre ellos, los DataNodes mandan su heartbeat a ambos servidores y sirven de testigos: el respaldo solo asume si la mayoría de DataNodes también dejó de ver al primario, y el primario, si no puede copiarle el journal al respaldo, sigue solo mientras la mayoría de DataNodes lo vea, y si no, deja de aceptar escrituras.

Figura 2

Diagrama de despliegue en AWS Academy



6. Bloques, metadatos y copias

6.1 Tamaño fijo de bloque

Todos los bloques miden 4 MB, y si el último queda incompleto se rellena y el tamaño real se guarda en los metadatos, para que `read()` nunca devuelva el relleno. Con un tamaño fijo, saber en qué bloque está un byte es una división (`bloque = offset / 4 MB`), el disco se reparte parejo y balancear los nodos es contar bloques; escogí 4 MB porque un archivo de 1 GB queda en 256 bloques, que alcanzan para descargar en paralelo sin llenar de entradas al control.

6.2 Metadatos

El control guarda en memoria, respaldado por el journal, un registro por archivo como este:

```
{ "path": "/users/miguel/videos/clase.mp4", "owner": "miguel",
  "size": 9437184, "version": 4, "block_size": 4194304, "state": "COMMITTED",
  "blocks": [
    { "idx": 0, "id": "b-7f3a-v4", "sha256": "9c1e...", "replicas": ["dn1","dn3","dn4"] },
    { "idx": 1, "id": "b-11c0-v2", "sha256": "a04b...", "replicas": ["dn2","dn4","dn1"] },
    { "idx": 2, "id": "b-5d22-v4", "sha256": "e77f...", "replicas": ["dn3","dn1","dn2"] } ] }
```

También guarda la lista de DataNodes con su dirección, espacio libre y último heartbeat, los locks activos y las cargas en curso. Un bloque modificado siempre recibe un id nuevo, y los ids nunca se reutilizan.

6.3 Ubicación de las copias y reconstrucción

Para cada bloque el control escoge 3 DataNodes distintos, empezando por los de más espacio libre y rotando el primero entre bloques seguidos, de modo que los bloques de un archivo, y también los distintos archivos, quedan repartidos entre todos los nodos (RNF4), como hacen GFS y HDFS (Ghemawat et al., 2003; Shvachko et al., 2010). Para reconstruir un archivo, el cliente pide el mapa, baja los bloques en paralelo, revisa el hash de cada uno, los une por orden y recorta al tamaño real.

7. Protocolos y APIs

El control usa REST sobre HTTPS con FastAPI, porque son mensajes pequeños, fáciles de probar y los códigos HTTP sirven para los errores; los datos usan gRPC sobre TLS, porque mandan binario, permiten enviar el bloque por partes (streaming) y el contrato queda en un `.proto` (gRPC Authors, s. f.). Todo se programa en Python, porque FastAPI y gRPC tienen librerías maduras en ese lenguaje y permiten tener un prototipo rápido, y cada proceso va en un contenedor Docker para que el mismo despliegue funcione igual en el portátil y en las EC2 de AWS Academy. No usé RabbitMQ, porque solo serviría para encolar la reposición de copias y el control ya puede mandar esas órdenes en la respuesta al heartbeat.

Tabla 3

API REST del servidor de control (prefijo /api/v1)

Método y ruta	Uso	Primitiva
`POST /auth/login`	Devuelve un JWT	—
`GET /fs/ls?path=`, `GET /fs/stat?path=`	Listar e información de una ruta	ls, cd
`POST /fs/mkdir`, `DELETE /fs/rmdir?path=`, `DELETE /fs/rm?path=`	Crear y borrar	mkdir, rmdir, rm
`POST /files/upload`, `POST /files/upload/{id}/commit`, `DELETE /files/upload/{id}`	Iniciar, confirmar o cancelar una carga	put
`GET /files/locate?path=&offset=&length=`	Bloques y copias vivas de un rango	get, read
`POST /files/open`, `POST /files/{fd}/close`	Abrir (fija la versión) y cerrar (confirma cambios)	open, close
`POST /locks`, `PUT /locks/{id}`, `DELETE /locks/{id}`	Pedir, renovar y soltar un lock	lock, unlock
`POST /files/{fd}/write`	Reserva bloques nuevos para un rango	write

Ejemplo de mensaje y formato de los errores:

```
POST /api/v1/files/upload
Authorization: Bearer eyJhbGci...
{ "path": "/users/miguel/datos.csv",
  "size": 9437184 }

201 Created
{ "upload_id": "up-91a2", "version": 1,
  "expires_in": 600,
  "blocks": [
   { "idx": 0, "id": "b-7f3a-v1",
     "targets": ["dn1:50051","dn3:50051",
                 "dn4:50051"],
     "token": "dGtuLmIt..." }, ... ] }
```

```
409 Conflict
{ "error": "VERSION_CONFLICT",
  "message": "versión base 3, actual 4",
  "retryable": false }

401 TOKEN_INVALID     403 FORBIDDEN
404 NOT_FOUND         409 VERSION_CONFLICT
400 INVALID_OFFSET    410 UPLOAD_EXPIRED
409 DIR_NOT_EMPTY
423 LOCKED (dice quién lo tiene
            y cuándo vence)
503 NOT_ENOUGH_NODES
```

Servicios gRPC de los DataNodes y del plano interno:

```
service DataNode {
  rpc PutBlock (stream BlockChunk) returns (PutReply);    // cliente o copia anterior
  rpc GetBlock (GetRequest) returns (stream BlockChunk);
  rpc DeleteBlock (DeleteRequest) returns (Ack);          // solo lo llama el control
  rpc ReplicateBlock (ReplicateRequest) returns (Ack);   // reposición de copias
}
service Control {
  rpc Heartbeat (HeartbeatReq) returns (HeartbeatReply); // cada 3 s, trae órdenes
  rpc BlockReport (BlockReportReq) returns (Ack);        // al arrancar y cada hora
}
service ControlReplica {                                  // servidor a servidor (S2S)
  rpc AppendJournal (JournalEntry) returns (Ack);         // copia cada cambio
  rpc Ping (PingReq) returns (PingReply);                 // cada 5 s
  rpc Snapshot (SnapshotReq) returns (stream Chunk);      // poner al día un respaldo
}
message JournalEntry { uint64 epoch = 1; uint64 seq = 2;
                       string op = 3; bytes payload = 4; }
message BlockChunk { string block_id = 1; bytes data = 2; string sha256 = 3;
                     string token = 4; repeated string next = 5; uint64 epoch = 6; }
```

Los errores gRPC son `UNAUTHENTICATED` (token de bloque inválido o vencido), `DATA_LOSS` (el hash no coincide), `RESOURCE_EXHAUSTED` (disco lleno), `FAILED_PRECONDITION` (época vieja) y `UNAVAILABLE` (la siguiente copia no responde). Entre los dos servidores de control, si el respaldo recibe un `JournalEntry` con un `seq` que no sigue al último que tiene, responde `OUT_OF_RANGE` y el primario le reenvía lo que falta, y si llega una época menor a la suya responde `FAILED_PRECONDITION`.

Figura 3

Secuencia de carga (put) con copia en cadena

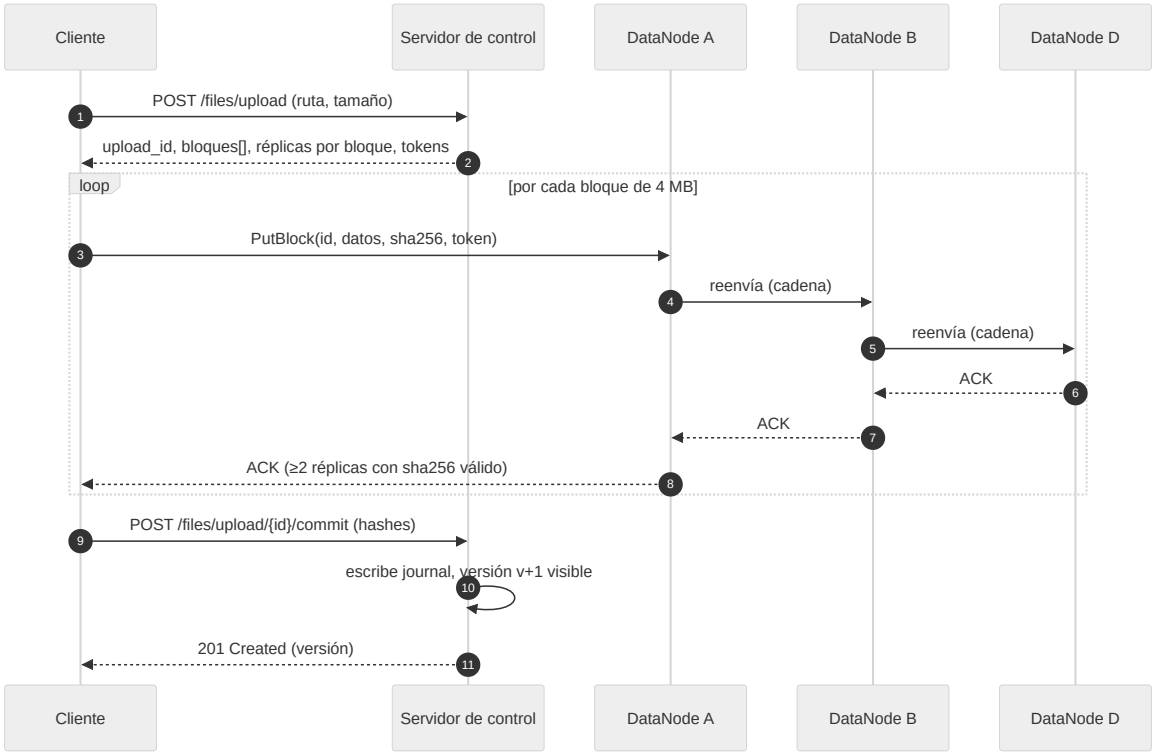
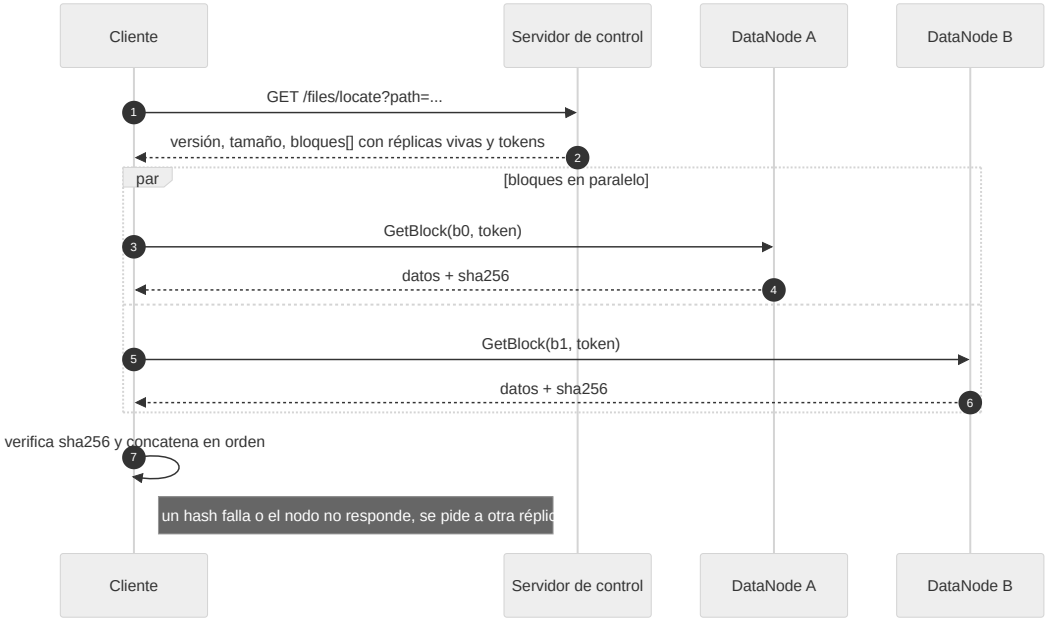


Figura 4

Secuencia de lectura (get o read) con descarga en paralelo



8. Semántica de read() y write()

Primero pensé en dejar cambiar cualquier byte en su sitio, como en un disco normal, pero eso obliga a coordinar las tres copias byte por byte y deja momentos en que un lector ve un bloque a medio cambiar; por eso escogí copy-on-write por bloque, que significa que un bloque confirmado nunca se sobrescribe, sino que se escribe uno nuevo y se cambia la referencia de una sola vez.

Tabla 4

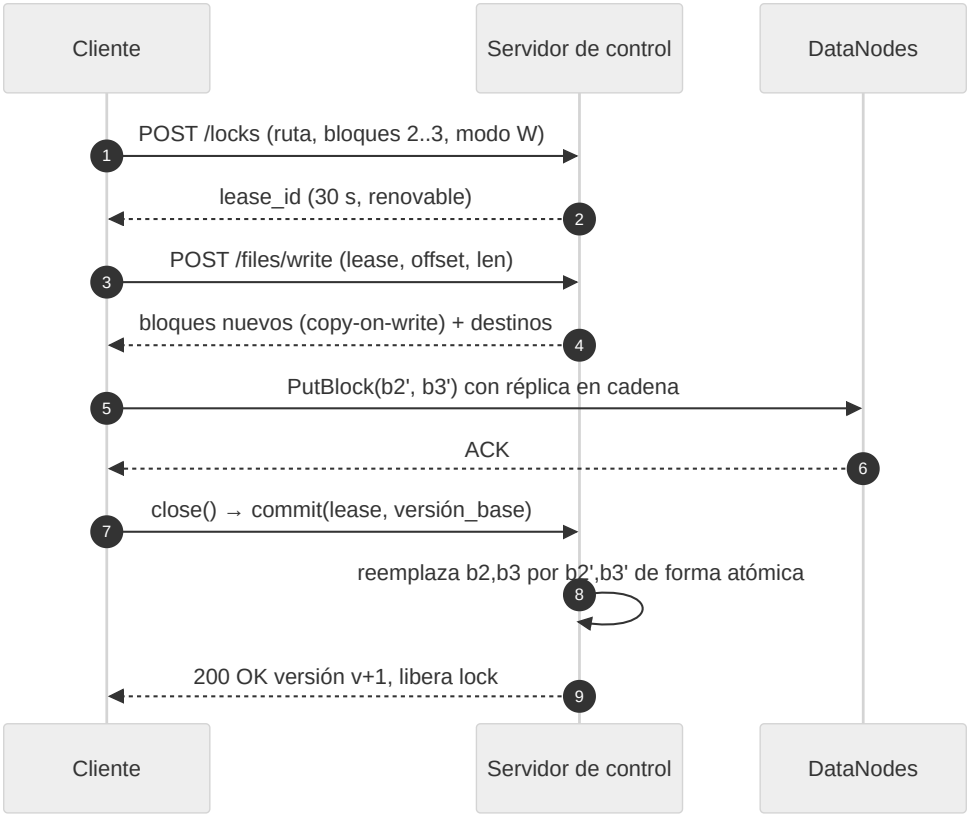
Semántica declarada de read() y write()

Aspecto	Decisión
Creación	Sí, con `put` o `open(path, CREATE)`.
Reemplazo completo	Sí, un `put` sobre un archivo existente crea la versión siguiente.
Anexado	Sí, un `write` con offset igual al tamaño actual, con lock sobre el último bloque. Un offset mayor al tamaño se rechaza con `400 INVALID_OFFSET`, porque dejaría un hueco.
Modificación parcial	Sí, en cualquier offset; por dentro se trabaja por bloque, el cliente lee el bloque, cambia los bytes y lo sube como bloque nuevo.
`read(fd, offset, len)`	Lee la versión fijada en `open()` y puede abarcar varios bloques en paralelo.
Visibilidad	De sesión: los cambios se ven cuando `close()` los confirma, y quien abrió antes sigue viendo su versión hasta que vuelva a abrir.
Concurrencia	Lectores sin límite y sin lock; escritores con lock exclusivo por rango de bloques, así que en bloques distintos avanzan a la vez y en el mismo bloque uno espera.

Garantía que se puede comprobar: una escritura está confirmada cuando el control responde `200` al commit, y en ese momento cada bloque nuevo tiene al menos 2 copias con el hash revisado y el cambio está en el journal del primario y del respaldo; todo `open()` posterior ve esa versión o una más nueva, y nadie puede leer una versión sin confirmar.

Figura 5

Secuencia de escritura parcial con lock y commit



9. Consistencia, concurrencia e integridad

Lock con lease. `lock(fd, bloque_ini, bloque_fin)` da exclusividad por 30 segundos, renovable cada 10, y si el cliente muere el lease vence solo, así que un proceso caído no deja el archivo bloqueado para siempre (Gray y Cheriton, 1989). Un `write()` sin lock vigente se rechaza, aunque la librería lo pide sola si el programador no lo hizo.

Versiones. Cada archivo tiene un número de versión, el commit lleva la versión base y el lease, y el control aplica los commits uno por uno, cambiando solo los bloques que cubre ese lease. El lock evita el conflicto y la versión lo detecta si algo falla, por ejemplo un lease que venció durante una pausa larga, y en ese caso el control responde `409` y el cliente vuelve a intentar.

Integridad. El cliente calcula el SHA-256 de cada bloque, cada DataNode lo revisa antes de confirmar y el cliente lo vuelve a revisar al leer. Los hashes detectan datos dañados, pero no coordinan a los escritores, por eso van junto con los locks y no los reemplazan.

Escrituras interrumpidas. Un bloque llega primero a una carpeta temporal del DataNode y solo cuando el hash es correcto se mueve con un `rename` atómico a la carpeta final; aun así, ningún bloque se ve hasta que un commit lo nombra. Si el cliente desaparece, la carga vence a los 10 minutos y los bloques sueltos se borran cuando el reporte del nodo muestra ids que ningún archivo usa. Si el control se cae justo después de guardar el commit y antes de responder, el cliente lo repite, y como el commit es idempotente, repetirlo con el mismo `upload_id` o lease devuelve el mismo resultado sin crear otra versión.

Bloques de versiones viejas. Cuando un commit reemplaza un bloque, el bloque anterior no se borra de inmediato, sino cuando ya ningún `open()` tiene fijada esa versión, o a más tardar después de 1 hora, para que los lectores que abrieron antes puedan terminar de leer.

10. Tolerancia a fallos y disponibilidad

Detección: cada DataNode manda un heartbeat cada 3 segundos; si pasan 10 sin recibirlo queda sospechoso y no recibe bloques nuevos, y a los 30 se da por caído.

Recuperación: cuando un nodo cae, el control busca sus bloques y ordena copiarlos desde una copia sana a otro nodo, empezando por los que tienen menos copias; si el nodo vuelve, su reporte permite borrar lo que sobre o esté viejo. Además, cada DataNode revisa cada cierto tiempo los hashes de sus bloques y reporta los dañados para recopiarlos.

Disponibilidad: con 3 copias y W = 2, las escrituras siguen con un DataNode caído y la lectura de un bloque funciona mientras quede una copia. Si cae el control primario, el respaldo toma su lugar en unos 15 segundos, y en ese tiempo no hay escrituras, pero las lecturas cuyo mapa ya tiene el cliente siguen funcionando.

11. Seguridad

Tabla 5

Mecanismos de seguridad

Aspecto	Mecanismo
Autenticación	Usuario y contraseña, guardada con bcrypt, a cambio de un JWT firmado que dura 1 hora (Jones et al., 2015).
Autorización	Toda ruta se resuelve dentro de `/users/<usuario>` y el control revisa el dueño en cada operación; para los datos entrega tokens de bloque (HMAC-SHA256 con id del bloque, operación y vencimiento de 5 minutos) que el DataNode valida sin consultar usuarios.

Cifrado en tránsito	TLS 1.3 entre cliente y control y entre cliente y DataNodes (Rescorla, 2018).
Comunicación interna	mTLS con una autoridad certificadora propia, así que sin certificado nadie puede mandar heartbeats ni copias; los Security Groups solo abren 443 y 50051 para el cliente y 50052 para el tráfico interno.
Cifrado en reposo	Cada DataNode cifra los bloques con AES-256-GCM (National Institute of Standards and Technology [NIST], 2007) con una clave que carga al arrancar y no está en el mismo disco; el journal se cifra igual y los volúmenes EBS se crean cifrados.

El límite que acepto es que alguien con acceso de administrador a un DataNode y a su clave podría leer sus bloques; cifrar en el cliente lo evitaría, pero complica el manejo de claves más de lo que necesita este proyecto.

12. Escenarios obligatorios

Dos procesos escriben el mismo bloque al mismo tiempo. Los dos piden lock sobre ese bloque y el control atiende las solicitudes en orden, así que el primero recibe el lease y el segundo recibe `423 LOCKED` con el tiempo que falta, y vuelve a intentar esperando cada vez un poco más. Cuando el primero hace `close()`, su versión queda confirmada y el lock se suelta, entonces el segundo lo obtiene, lee el bloque ya actualizado y escribe encima, creando la versión siguiente. No se mezclan bytes, porque cada escritor sube un bloque nuevo completo, y no se pierde ningún cambio, porque el segundo parte de lo que dejó el primero.

Una réplica está desconectada durante una escritura. Si el tercer nodo de la cadena se desconecta justo durante la escritura, el segundo no puede pasarle el bloque y confirma con dos copias (si el que falla es el del medio, el primero salta al siguiente de la lista `next`), y como W = 2 la escritura se acepta y el bloque queda marcado como sub-replicado. Los lectores no notan nada, porque solo ven versiones confirmadas y el mapa solo muestra copias vivas. Gracias al copy-on-write, el nodo caído nunca tiene un bloque viejo con el id nuevo, así que cuando vuelve sus bloques viejos no los usa nadie y se borran, y el control ordena copiar el bloque nuevo hasta tener 3 copias. Si fallan dos de los tres nodos, no se llega a W = 2 y el cliente pide otros destinos o recibe `503`.

Un nodo falla después de recibir parte de un bloque. Ese pedazo queda en la carpeta temporal, sin hash revisado y sin confirmación, el cliente detecta el corte (`UNAVAILABLE` o timeout), le pide al control otro destino y manda el bloque completo de nuevo. El commit solo se acepta si todos los bloques tienen al menos dos copias con hash correcto, y si falta alguno el archivo sigue en su versión anterior o no existe, de modo que publicar un archivo incompleto no es posible, porque lo que lo hace visible es

un único cambio atómico en los metadatos y no el estado de cada nodo. Los restos temporales se borran cuando vence la carga.

El cliente no conoce la ubicación de los bloques. El cliente solo conoce el nombre `control.dfsha.internal`, que el DNS interno resuelve al primario de turno, y con `GET /files/locate` recibe, para cada bloque, los DataNodes que lo tienen y su dirección actual, que cada nodo registra en sus heartbeats, así que un nodo puede cambiar de IP sin afectar a nadie. El cliente guarda ese mapa 60 segundos, y si una copia falla prueba la siguiente, y si fallan todas vuelve a preguntar; ninguna dirección de DataNode está escrita en el cliente (RNF7).

13. Métricas y experimentos

Tabla 6

Experimentos previstos

#	Experimento	Variable	Qué se mide	Qué espero
E1	Subir y bajar archivos de 10 MB, 100 MB y 1 GB, y comparar con `sha256sum`	Tamaño	Latencia, MB/s y que el archivo sea idéntico	Tiempo casi proporcional al tamaño y hash igual siempre
E2	Leer con 1, 2, 4 y 8 descargas en paralelo	Paralelismo	Throughput de lectura	Más MB/s hasta llenar la red de la instancia
E3	De 1 a 8 clientes escribiendo bloques distintos y el mismo bloque	Concurrencia	Latencia de write, espera por lock y versión final correcta	Bloques distintos sin espera; el mismo bloque se serializa
E4	Apagar un DataNode durante lecturas y escrituras	Fallo de nodo	Operaciones fallidas, tiempo de detección y de reposición	Ninguna lectura falla; las escrituras siguen con W = 2
E5	Escribir con 2 copias y con 3 copias	Replicación	Latencia de put y espacio usado	R = 3 algo más lento y 50 % más espacio
E6	Throughput total con 4 y con 6 DataNodes	Escalabilidad	MB/s con 8 clientes	Throughput sube casi en proporción a los nodos
E7	Apagar el control primario	Fallo del control	Tiempo sin escrituras hasta que el respaldo asume	Unos 15 s sin escrituras; las lecturas siguen

Cada prueba se repite al menos 5 veces con un script en Python y se reporta la mediana y el percentil 95, porque en una red compartida como la de AWS Academy un promedio solo puede engañar.

14. Cómo se cumplen los requisitos no funcionales

Tabla 7

Relación entre los requisitos no funcionales y el diseño

Requisito	Cómo lo cumple el diseño	Sección
RNF1. Escalabilidad	Usuarios: el control no guarda sesiones porque usa JWT. Tamaño de archivos: un archivo grande solo suma bloques repartidos entre nodos. Número de archivos: cada bloque ocupa unos 200 bytes de metadatos, así que 1 TB de datos (262.144 bloques) cabe en unos 50 MB de memoria del control. Más capacidad o velocidad es agregar DataNodes con `add-node`.	5, 6, 13 (E6)
RNF2. Disponibilidad	3 copias por bloque, control con respaldo, detección por heartbeats y reposición automática de copias.	5, 10
RNF3. Consistencia	Commit atómico en el control, versiones, semántica de sesión y garantía comprobable de la sección 8.	8, 9
RNF4. Particionamiento	Bloques fijos de 4 MB, cada uno en 3 nodos distintos, rotando el nodo inicial.	6
RNF5. Rendimiento y concurrencia	Los datos no pasan por el control, se descargan bloques en paralelo y los locks son por rango de bloques, no por archivo completo.	5, 8, 9
RNF6. Seguridad	JWT, permisos por dueño, tokens de bloque, TLS 1.3, mTLS interno y AES-256-GCM en reposo.	11
RNF7. Transparencia	Acceso: el usuario usa los mismos comandos sin importar en qué nodos esté el archivo. Localización: el cliente solo conoce un nombre DNS y descubre los nodos con `locate`.	7, 12

15. Compromisos técnicos

Control central o distribuido: un solo control con respaldo es fácil de entender y da un orden único a los commits, pero su memoria limita cuántos archivos caben, por eso los bloques son de 4 MB y no más pequeños, y el cambio al respaldo deja unos segundos sin escrituras.

W = 2 o W = 3: esperar las tres copias es más seguro en el momento, pero un solo nodo lento frenaría todas las escrituras, y con W = 2 se aguanta un fallo y la tercera copia se completa después.

Copy-on-write o escribir en el sitio: cambiar un byte cuesta reescribir 4 MB, pero a cambio los commits son atómicos, los lectores no se bloquean y ninguna copia queda a medio cambiar.

Semántica de sesión o POSIX: los cambios se ven al cerrar y no con cada `write()`, lo que es menos inmediato, pero es una garantía que se puede probar y que encaja con el uso normal de archivos grandes (Tanenbaum y Van Steen, 2023).

16. Conclusión

Guardar bytes en varias máquinas no es lo difícil; lo difícil es decidir cuándo una escritura ya pasó, y todo este diseño gira alrededor de esa respuesta: un cambio existe cuando el control lo confirma, y antes de eso nadie lo ve. Los bloques del mismo tamaño, el copy-on-write, los locks con lease y las dos copias mínimas son formas distintas de cumplir esa misma garantía, y lo que falta, como el tamaño ideal del bloque o cuánto tarda de verdad el cambio al respaldo, se tendrá que ajustar en la fase de equipo con mediciones y no con supuestos.

Referencias

Ghemawat, S., Gobioff, H. y Leung, S.-T. (2003). The Google File System. *Proceedings of the 19th ACM Symposium on Operating Systems Principles*, 29–43. https://doi.org/10.1145/945445.945450

Gray, C. y Cheriton, D. (1989). Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. *Proceedings of the 12th ACM Symposium on Operating Systems Principles*, 202–210. https://doi.org/10.1145/74850.74870

gRPC Authors. (s. f.). *gRPC documentation*. https://grpc.io/docs/

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF. https://www.rfc-editor.org/rfc/rfc7519

National Institute of Standards and Technology. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (SP 800-38D). https://doi.org/10.6028/NIST.SP.800-38D

Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). IETF. https://www.rfc-editor.org/rfc/rfc8446

Shvachko, K., Kuang, H., Radia, S. y Chansler, R. (2010). The Hadoop Distributed File System. *2010 IEEE 26th Symposium on Mass Storage Systems and Technologies*, 1–10. https://doi.org/10.1109/MSST.2010.5496972

Tanenbaum, A. S. y Van Steen, M. (2023). *Distributed systems* (4.ª ed.). distributed-systems.net.